AlpaBridge: Separating Integration Failures from Policy Failures in Closed-Loop Driving Simulation

Alba Maria Tellez Fernandez

***Abstract*— Closed-loop driving evaluation can misattribute degraded behavior to policy quality when an adapter loses decision-relevant inputs. AlpaBridge audits semantic, temporal, lifecycle, deployment, and evidence contracts before simulator output is admitted as policy evidence. We test attribution correctness with a specified 2×2 negative-control design on the 20 WOMD TFExamples bundled with a pinned Waymax revision; 19 contain valid route geometry. For each, the same pure-pursuit policy receives either original WOMD geometry or a degraded geometric proxy reconstructed from an intervention-defined command. A constant-velocity policy traverses the same mutation but does not consume route input. All four arms complete 50-step rollouts. Route-following endpoint divergence has median 1.017 m and exceeds 0.1 m in 13/19 scenarios, whereas constant velocity is numerically invariant in 19/19. AlpaBridge rejects only the 19 route-dependent proxy executions as `semantic.command_only`. A separate suite of 15 clean and 15 single-fault adapter sessions yields 30/30 classifications, 15/15 localizations, and 0/15 control false positives. Simulator completion alone therefore cannot distinguish policy degradation from adapter-induced information loss. The study does not establish policy superiority, representative WOMD performance, safety, fault prevalence, or comparative runtime overhead.**

I. Introduction

Dataset-trained trajectory policies and closed-loop simulators expose different failure surfaces. A trajectory policy developed from the Waymo Open Motion Dataset (WOMD) setting [1] may assume logged agent and map observations, a declared intent representation, and a fixed trajectory horizon. A simulator service must also accept incremental messages, meet the runtime clock, remain alive across session events, and retain enough evidence to explain a result. AlpaSim exposes this difference through its external-driver boundary [2].

A wrapper can therefore make an integration defect look like poor driving. Replacing a local route polyline with a left/straight/right command removes decision-relevant input, yet the rollout may still terminate with collision and progress metrics. Timing-grid mismatches, stale observations, optional import failures, missing assets, and incomplete manifests create similar confounds. A completed process is not sufficient evidence that a score characterizes policy behavior.

AlpaBridge treats this as a system-integration and attribution problem. Its contribution is threefold:

- five explicit contracts define when a simulator rollout may be interpreted as policy behavior;
- an adapter, audit, and artifact pipeline localizes violations and preserves completed, blocked, and invalid rows;
- a policy-by-route negative-control experiment over real WOMD records tests whether a detected semantic violation selectively changes a policy that requires the lost information, while a protocol-conformance suite and AlpaSim execution test localization and external-driver operation.

The five contract classes concern information, clocks, service state, deployment, and evidence rather than WOD records or a specific protobuf schema. A new binding maps its native policy inputs, clocks, service states, and artifacts to these contracts; the WOD-style/AlpaSim adapter is one instantiation, not the definition. Empirically, this paper exercises the semantic rule in both Waymax and AlpaSim. The Waymax experiment is a bounded WOMD-native binding for one route contract, not a production adapter or cross-runtime validation of all five contracts; the AlpaSim matrix uses dependency-light policies, while a replay and one reactive run add a published NAVSIM learned baseline.

A. Failure Attribution Rule

A rollout permits policy-behavior attribution only if its semantic, temporal, lifecycle, deployment, and evidence contracts pass. A policy-failure attribution additionally requires a retained failure layer identifying the policy. Otherwise the row is an integration, precondition, or evidence failure. Thus a missing actor proxy is not a weak planner, a command-proxy route is not a poor route-conditioned policy, and an incomplete audit is not a collision result. This rule permits policy behavior to be examined after validation; it does not declare every degraded metric a policy failure.

II. Contract-Gated Integration

Table I maps the five recurring boundary mismatches to enforcement mechanisms. The *semantic contract* preserves the inputs declared by the active policy, including route geometry for route-conditioned policies plus required ego, camera, and actor fields at prediction time. A high-level route command cannot substitute for a local polyline when geometry is declared. The *temporal contract* validates horizons and timestamps, anchors interpolation at the current ego pose, deterministically resamples a finite $N \times 2$ trajectory, and recomputes headings on the runtime grid.

The *lifecycle contract* makes start, active, close, cleanup, duplicate close, and late events explicit. Recoverable warnings are distinguished from fatal service errors. The *deployment contract* records launch commands, scene and policy identifiers, timeout, endpoint, repository state, image identity, and gated inputs. The *evidence contract* requires unique

Alba Maria Tellez Fernandez is an Independent Researcher. Email: amtellezfernandez@gmail.com.

TABLE I

CONTRACTS ENFORCED AT THE POLICY/SIMULATOR BOUNDARY.

Mismatch	Contract	Mechanism	Validation
Command-only route	Semantic	Preserve route geometry	route-source audit
Policy horizon/runtime grid	Temporal	Deterministic resampling	cadence tests
Script flow/session service	Lifecycle	Idempotent late-event handling	lifecycle tests
Implicit host state	Deployment	Materialized manifests	readiness checks
Process exit/evidence	Evidence	Audit-valid summaries	claim gate

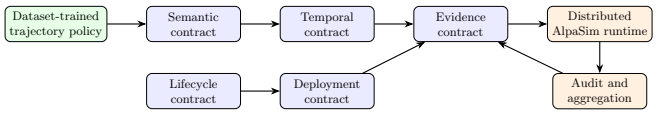

Fig. 1. Contract gates localize semantic, temporal, lifecycle, deployment, and evidence failures around the external-driver boundary.

run identifiers, immutable manifests, terminal status, route and sensor audits, failed-run retention, and hashes linking generated paper assets to aggregate rows.

For rollout r and policy signature p, admission is

$$\mathcal{A}(r,p) = \bigwedge_{c\in\{\mathrm{sem},\mathrm{temp},\mathrm{life},\mathrm{deploy},\mathrm{evid}\}} C_c(r,p). \quad (1)$$

A behavior measurement $Q(r)$ is policy-behavior-attributable only when $\mathcal{A}(r,p)=1$; a policy-failure claim additionally requires the retained failure layer to be `policy`. Contract checks consume declared requirements, message provenance, timestamps, lifecycle events, and artifacts, not $Q(r)$. Admission is therefore independent of whether the resulting trajectory appears good or bad.

Figure 1 shows the boundary. AlpaBridge reconstructs policy-facing state, preserves route geometry, resamples outputs, and isolates optional model discovery. On the runtime side it records deployment and evidence state before a row can be attributed to policy behavior. Constant-velocity and route-following models form the public core. The protocol replay downloads and hash-checks NAVSIM's official EgoStatusMLP seed-0 checkpoint without redistributing it. Other learned checkpoints and direct actor-aware entry points remain gated when their artifacts or scene-matched actor proxies are unavailable.

III. IMPLEMENTATION AND EVIDENCE PIPELINE

A. Policy-Facing Adapter

The external driver maintains isolated state for each simulator session. Image, egomotion, route, and close messages update that state; prediction reads one consistent snapshot rather than reconstructing inputs from process-global variables. The route converter carries simulator waypoints into the policy input and records their source. When the explicit command-only diagnostic is selected, it labels the fallback as `command_proxy`; the audit rejects the row if the active policy declares route geometry as required, even if the service and simulator finish normally. A command-native policy does not produce that false alarm. Static hazards remain geometry-bearing objects, while dynamic hazards retain position, motion, heading, and shape fields when the external interface supplies them. Visibility-risk signals remain diagnostics and do not create synthetic obstacles.

The temporal adapter maps a source trajectory to runtime times $t_k = k/f_{\mathrm{run}}$ over the requested horizon. The current ego origin is prepended before interpolation, source timestamps must be finite and strictly increasing, and the output must be a finite $N \times 2$ array. Matching grids preserve samples; mismatched grids use deterministic interpolation, after which headings are recomputed from successive runtime points. The final protocol serializer independently rejects empty trajectories, non-finite coordinates or headings, and unequal point/heading counts. Invalid horizons, frequencies, grids, or trajectories therefore fail the temporal contract rather than reaching vehicle control. These behaviors are covered by dependency-light tests, but the scene-level temporal comparison remains blocked and is not reported as a result.

B. Artifact Lineage

Before launch, the matrix runner records command, environment, scene and policy identities, adapter mode, prerequisites, and expected route source. Readiness and terminal states remain in the denominator. For completed rows, the audit reconstructs route and freshness status from telemetry before loading behavior metrics. Aggregation rejects duplicate run identifiers, preserves failure codes, and pairs rows by policy, scene, and execution. CSV, JSON, tables, and figures share one aggregate-data hash that the submission validator checks. Licensed scenes and sensor frames are not redistributed, so public lineage is complete while replay still requires legitimately acquired inputs and runtime software.

IV. EVALUATION METHOD

The contract-validation matrix (CVM) contains 148 configured rows: 45 core, 30 semantic-ablation, 18 temporal-ablation, 40 lifecycle, and 15 fault-injection rows. It completes 115 rows, including 60 simulator rollouts; 33 optional-extension rows remain blocked.

A. Policy-Dependency Negative Control

Our primary question is whether the audit can distinguish degradation caused by a lost policy input from genuine policy failure. We use all 20 TFExamples in the route-bearing test fixture distributed with a commit-pinned Waymax checkout [3]; raw records are not redistributed. One example has no valid `sdc_paths.on_route` candidate and is excluded before intervention, leaving 19 paired WOMD scenarios.

The design crosses policy $P \in \{\mathrm{RF},\mathrm{CV}\}$ with route representation $R \in \{\mathrm{full},\mathrm{proxy}\}$. RF is a constant-speed, no-jump pure-pursuit controller. In the full arm it receives the selected WOMD route polyline. In the proxy arm, the adapter removes that original geometry and constructs a straight 200-m polyline from an intervention-defined `KEEP_HEADING` command. This command is not read from the WOMD record. Both arms retain the same control law, 0.5-s lookahead, 2-m minimum lookahead, and current-state speed; only the received geometry changes. CV traverses the same adapter

mutation but its declared signature excludes route, so it ignores both representations.

Every arm starts from the same state and uses Waymax `StateDynamics` for 50 steps at 10 Hz; horizon, controlled SDC, and log playback of other agents are fixed. Thus the experiment comprises 3800 closed-loop steps and 3800 finite plans. For scenario i and policy p, the paired divergence is

$$D_i^p(t) = \left\| x_{i,t}^{p,\text{full}} - x_{i,t}^{p,\text{proxy}} \right\|_2, \tag{2}$$

and the primary policy-by-route interaction is $I_i = D_i^{\text{RF}}(T) - D_i^{\text{CV}}(T)$. Before stepping or computing behavior metrics, the audit evaluates the declared route requirement and source. Before the retained corrected run, we fixed three checks: H1, median $D^{\text{RF}}(T) > 0.1$ m; H2, every $D_i^{\text{CV}}(T) \leq 10^{-6}$ m; and H3, only RF/proxy receives `semantic.command_only`. An exploratory prototype on the same fixture had substituted a straight-line controller in the proxy arm; it exposed that confound and is not retained. The checks are therefore deterministic fixture-level decision criteria, not an independent preregistration or population significance test. Collision and overlap are retained as descriptive Waymax outputs but are not primary because non-SDC agents follow the log and cannot react to the counterfactual ego.

B. Waymax Reproducibility

The repository entry point installs the pinned upstream revision in an isolated environment, verifies the bundled fixture hash, and executes each eligible scenario in all four arms. Eligibility is decided before route intervention; failed arms remain in the scenario record rather than disappearing from a denominator. The retained package contains a manifest, per-scenario JSON Lines, an aggregate JSON summary, implementation hashes, dependency versions, and the exact CPU command. The paper aggregator recomputes all endpoint and audit counts from those files and includes their content hash in every generated table and figure. The submission validator compares the LaTeX macros with the JSON source. Raw upstream TFRecords are intentionally omitted; reproducing the run therefore requires the pinned Waymax checkout and acceptance of its licensing terms.

The public AlpaSim core executes two dependency-light policies on 15 locally cached scenes and pairs full-route and command-only route-following arms. Each arm has one execution because the launcher exposes no seed override. Scene categories are unverified (0/6), so these rows are integration diagnostics, not coverage evidence.

For transport-inclusive evidence, one official AlpaSim integration recording is replayed through four live services: route following and NAVSIM's published EgoStatusMLP seed-0 baseline [4], [5], each with full geometry or command only. The CPU checkpoint is revision- and hash-pinned. EgoStatusMLP consumes ego status and command, not pixels or the polyline. Each sequential arm has 60 paired calls. Returned trajectories cannot affect recorded future observations, so this is non-reactive protocol replay, not a rollout or an overhead comparison.

One separate run tests live feedback with the same checkpoint and public AlpaSim fixture: external driver, controller, and physics remain live, and each trajectory changes the next ego state. The declared fixture adds a flat collision surface and repeats a seed frame at AlpaSim's `video_model` boundary. This is admissible only for the camera-blind checkpoint; a camera-requiring control must reject the stale frame.

The retained external trace has 197 calls but predates the current finite-output field. The mutation study instead uses 15 current-schema clean sessions, all contract-valid before mutation. Each is paired with one independently scored semantic, temporal, lifecycle, deployment, or evidence mutation; the detector sees events and runtime context but not the expected label. For a paired comparator, define the executable completion-and-metrics gate

$$B_{\text{status}}(r) = \mathbf{1}[\text{completed}(r) \wedge \text{metrics}(r)]. \tag{3}$$

Both methods return a decision for all 30 cases. We report exact classification, detection, localization, false-positive, and paired-discordance counts. We do not compute a population confidence interval or hypothesis test because the sessions and fault operators are deliberately constructed rather than sampled independently from a defined population.

Randomized microbenchmarks time post-parse detection and a paired in-process Drive path with camera and freshness guards enabled or disabled; they exclude I/O, gRPC, simulator work, and human investigation. Guarded and unchecked outputs must match exactly. Completed simulator rows are separately audited for route source and freshness.

V. Results

Figure 2 reports the primary causal check. All four arms complete for 19 scenarios. RF has endpoint divergence mean 1.973 m, median 1.017 m, and maximum 14.211 m; 13 scenarios exceed 0.1 m and 10 exceed 1 m. CV is numerically identical in the retained traces: its maximum endpoint divergence is 0.000 m, and all 19/19 satisfy the 10^{-6}-m invariant. The interaction therefore has mean 1.973 m and median 1.017 m. H1 and H2 hold for the pinned fixture.

Table II separates completion from attribution. The audit accepts all 19 RF/full executions and rejects all 19 RF/proxy executions with only `semantic.command_only`. It accepts both CV arms (19/19 and 19/19), because geometry is outside that policy's signature. H3 therefore holds. Importantly, the rejected RF/proxy arms still finish and emit Waymax metrics: invalidation does not follow from poor trajectory values, and no policy-ranking or safety conclusion is drawn from collision or overlap.

All 30/30 dependency-light public-core rows complete without service crash or blocker, but only 28 are audit-valid. Late command-proxy fallback makes the other completed rows integration/evidence failures rather than policy evidence.

Designed-suite results appear in Table III. AlpaBridge is correct on 30 of 30 cases, detects and localizes all 15 faults, and flags none of the 15 controls. The status gate is correct on 15 cases because it accepts every control and every completed fault case; it detects no faults. All 15 discordant pairs favor

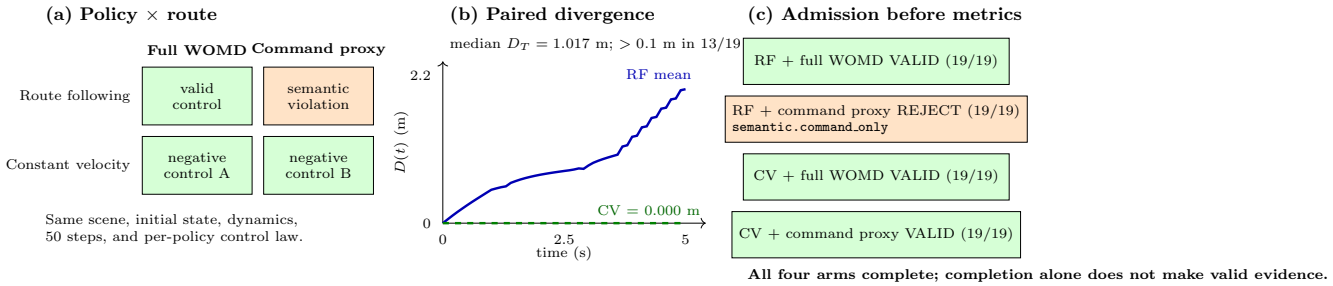

Fig. 2. Policy-dependency negative control on WOMD scenarios in Waymax. The same route-following controller receives full route geometry or an intervention-defined command proxy; constant velocity traverses the same adapter mutation without consuming route. Mean paired divergence grows only for the route-dependent policy. Audit admission is decided from the declared input signature before behavior metrics.

TABLE II

WAYMAX/WOMD POLICY-BY-ROUTE RESULT. D_T COMPARES FULL AND PROXY ENDPOINTS WITHIN THE SAME POLICY; "VALID" IS CONTRACT ADMISSION, NOT DRIVING QUALITY.

Policy	Median D_T (m)	$D_T > 0.1$ m	Full valid	Proxy valid
Route following	1.017	13/19	19/19	0/19
Constant velocity	0.000	0/19	19/19	19/19

TABLE III

PAIRED PROTOCOL-CONFORMANCE DIAGNOSTICS. FP DENOTES A VALID CONTROL CLASSIFIED AS FAULTY.

Gate	Correct	Detected	Localized	FP
AlpaBridge	30/30	15/15	15/15	0/15
Status-only	15/30	0/15	0/15	0/15

TABLE IV

EXECUTED ALPASIM POLICY-FAMILY PROTOCOL REPLAY. "MOVING" MEANS ENDPOINT PROGRESS EXCEEDS 1 M; AUDIT OUTCOMES ARE RECOMPUTED FROM RETAINED TELEMETRY.

Policy	Policy-visible route	Finite	Moving	Audit
Route following	20 waypoints	60/60	60/60	pass
Route following	command only	60/60	60/60	semantic fault
NAVSIM EgoStatusMLP	command (route retained)	60/60	60/60	pass
NAVSIM EgoStatusMLP	command only	60/60	60/60	pass

AlpaBridge. These are descriptive counts for the declared suite, not evidence of population-level superiority to another integration framework.

Post-parse detection is 26.169 μs median and 52.962 μs p95; the paired guard-path increment is 68.871 μs median and 309.613 μs p95. Outputs are identical between timed paths. These one-host microbenchmarks are not end-to-end runtime or time-to-diagnosis measurements.

All four protocol-replay arms return 60 finite, moving trajectories. Full-route traces pass. Command-only route following receives only `semantic.command_only` and changes endpoint on 56/60 calls; all 60 EgoStatusMLP pairs are exactly equal and pass because route geometry is not a model input. This is input-signature evidence, not format or policy overhead.

The learned reactive arm completes its one rollout with 197/197 finite outputs and advances 61.863 m over 19.930 simulated seconds. These are measurements of that exact configuration, not overhead versus another integration or a policy-quality result. The camera-requiring control completes 4 calls, then rejects the next unchanged frame as stale.

The external runs also exposed three non-injected defects: stale camera-alias priority, zero dynamic speed despite advancing poses, and a trajectory-serializer off-by-one. Freshest-frame selection, pose-derived speed fallback, and current/future count checks now have regression tests. All replay arms were regenerated; these repairs are not counted among the designed faults.

A. Cross-Runtime Scope

The unit of transfer is a declared contract binding, not a dataset conversion. In Waymax, the policy-side representation is native WOMD geometry and the intervention changes the closed-loop SDC trajectory while other agents follow the log. In AlpaSim, the external-driver binding independently records whether a policy receives geometry or only a command. The replay then applies that rule to a route-dependent controller and a published learned command-native policy; the live run adds simulator feedback for the learned service. Across both runtimes, command-derived geometry is rejected only when the active signature requires the original polyline.

This is mechanism-level replication of one semantic rule under different state, dynamics, and service interfaces. It is not simulator equivalence or policy-performance transfer: the two route-following implementations are not asserted to be identical, the learned checkpoint is NAVSIM rather than WOMD-trained, and only Waymax supplies the paired causal rollout. Testing the same learned WOMD policy in both runtimes and repeating the other four contracts remain required for broader generalization.

All 15/15 metric-bearing command-only rows satisfy B_{status}, whereas AlpaBridge rejects 15/15 as route-invalid. Their simulator outcomes cannot be used as a policy effect because each AlpaSim arm has one execution and no effective seed control.

Across all retained rows, 42 are policy-behavior-attributable diagnostics, 0 are policy-failure-attributable, and 33 are integration/precondition blockers. The remaining 106 include blocked, contract-invalid, synthetic, or benchmark-incomplete

evidence. These counts operationalize the distinction: degraded metrics cannot assign policy failure unless the contracts pass and the retained failure layer is policy.

B. Interpretation

The controlled experiment validates classification and localization for the declared mutation set. Its 15 separately instantiated clean sessions supply the paired negative cases, so the observed 0/15 false-positive count is scored from detector output rather than defined by audit admission. The sessions and mutations are nevertheless products of the same protocol design and are not independent population samples. The closed-loop rows test a second path: adding route-source validity changes the attribution of all 15 metric-bearing command-only records without changing their simulator outcome.

The timing microbenchmarks instrument two software paths. Within the in-process adapter, camera-set and freshness checks have a median increment of 68.871 μs. Post-parse detector execution has a median of 28.096 μs. A missing actor proxy or checkpoint remains a deployment precondition, a command proxy a semantic violation, and missing audit state an evidence violation. Typed outcomes keep those cases available for debugging while excluding them from policy attribution.

VI. Related Work and Limitations

The contracts instantiate interface reasoning for cyber-physical systems [6], [7]. Closed-loop platforms and benchmarks such as CARLA, nuPlan, Waymax, NAVSIM, and AlpaSim [8], [9], [3], [4], [2] emphasize simulation or planning evaluation. Scenario and falsification tools such as Scenic, VerifAI, DriveFuzz, and PlanFuzz [10], [11], [12], [13] generate or mutate conditions. AlpaBridge is complementary: it asks whether the policy/simulator boundary preserved the assumptions needed to interpret a rollout.

The abstraction boundary is broader than WOD: a route-conditioned policy in another framework can bind its intent representation to the semantic contract, its prediction and control clocks to the temporal contract, and its service and artifact states to the remaining contracts. Such a binding may reuse the audit vocabulary without sharing WOD message types. The route-following treatment and EgoStatusMLP negative control show that the semantic gate follows a policy's declared input signature rather than one policy implementation. The factorial experiment additionally executes that rule in a WOMD-native Waymax environment, while the external-driver evidence uses AlpaSim. This is cross-runtime evidence for the route semantic contract, not evidence that all five contracts, policies, or fault classes transfer unchanged between the frameworks.

nuPlan and NAVSIM formalize planning tasks and measures; Waymax and CARLA provide controllable simulation interfaces; AlpaSim supplies the external-driver boundary used here. AlpaBridge instead applies contract assumptions and guarantees as runtime admission and artifact-lineage checks. Scenic and driving fuzzers vary scenarios to expose system behavior, whereas our mutations vary information or preconditions at the integration boundary. The approaches can be combined.

The study scope is contract conformance. The diagnostic mutations and clean sessions are framework-authored, deterministic software cases rather than natural fault samples. The retained external trace establishes a separate service-conformance result, but its earlier schema omits the explicit finite-output field required by the current detector. The status-only gate is a concrete completion-and-metrics comparator, not a surrogate for CARLA, nuPlan, or another full framework. The Waymax study uses the complete 20-example route test fixture bundled by the pinned upstream revision; it is neither a random nor representative WOMD sample, and one example lacks the route required for the paired comparison. It evaluates two deterministic policies, not a learned WOMD planner. AlpaBridge does not redistribute the fixture or reconstruct WOMD worlds inside AlpaSim, and the AlpaSim matrix has no scene-matched actor proxy or verified scenario-category coverage. One official NAVSIM learned checkpoint is hash-pinned for the replay and one reactive external-driver rollout, but is not redistributed. It is camera-blind rather than visual or multimodal and is not evaluated for policy quality. The public reactive fixture repeats a recorded seed frame and uses a declared synthetic flat physics surface. Restricted scenes remain licensed inputs; public artifacts retain telemetry, manifests, aggregates, hashes, and testable gate logic. The study does not measure how often route geometry or another policy input is lost in production integrations; the designed ablation establishes consequence and detection for the executed boundary only.

A. Threats to Validity

Internal validity. Mutation and detector code share the declared contracts, which can favor the expected fault vocabulary. To reduce circularity, construction and detection are separate functions, expected labels are withheld from the detector, every case is scored afterward, and 15 unmutated sessions test false alarms. The exact paired counts describe only this designed set; no independence-based significance test is applied. Timing order is randomized and paired on one host, so hardware and background-load effects remain shared measurement conditions.

Construct validity. The guard-path ablation covers camera-set and freshness checks in the in-process adapter Drive path over 15 deterministic valid sessions. It includes input assembly, prediction, serialization, finite-output validation, reasoning parsing, and in-memory telemetry, but excludes gRPC, file I/O, simulator work, and human investigation. The detector timer begins after JSON parsing and excludes I/O and human investigation. Neither microbenchmark timer supports an end-to-end runtime or time-to-diagnosis claim. The recorded-message replay adds gRPC transport and service dispatch, but excludes simulator stepping and feedback; it therefore supports client-to-service latency, not end-to-end runtime or human diagnosis time. The separate learned rollout includes simulator stepping and feedback and therefore supports the reported duration and service timing for that con-

figuration. Its repeated camera seed and flat ground exclude reactive visual behavior, rendering quality, and comparative overhead. All five contracts, including the plugin precondition within deployment, are mutated against generated telemetry and context rather than rerun as physical simulator faults. Contract-valid therefore means admitted to the stated evidence boundary, not driving safety or simulator correctness.

External and conclusion validity. The AlpaSim scenes were selected by local asset availability and remain category-unclassified. The Waymax fixture is small and upstream-selected for testing, not evaluation coverage. Its paired arms are deterministic and share initial state, but log-playback agents do not react to the counterfactual ego. Accordingly, endpoint divergence supports selective behavioral consequence, while collision and overlap do not establish counterfactual safety. The learned replay measures input-signature invariance under route reduction; the reactive learned run covers one scene with synthetic ground and non-reactive pixels. None establishes policy quality, fault prevalence, or cross-dataset generalization. The semantic route rule is exercised in two independently maintained runtimes, but cross-runtime generalization of the other four contracts and learned WOMD policies requires further experiments.

VII. Conclusion

AlpaBridge formulates a closed-loop policy boundary as semantic, temporal, lifecycle, deployment, and evidence contracts. In the WOMD-native factorial experiment, degrading route geometry selectively changes the route-dependent controller (median interaction 1.017 m), while the route-independent control remains exactly invariant on 19/19 scenarios. All arms complete, yet the audit rejects only RF/proxy. This establishes the central attribution result: completion is insufficient to decide whether a metric characterizes policy quality. On 15 designed faults paired with 15 valid sessions, AlpaBridge classifies 30/30 and localizes 15/15. AlpaSim replay and reactive execution provide complementary external-driver evidence, so the semantic route rule is exercised in two runtimes. Representative WOMD performance, learned-WOMD-policy transfer, safety, comparative overhead, human diagnosis time, and framework superiority remain outside the claims.

Acknowledgment

OpenAI Codex (GPT-5) assisted with language editing, code audit, and test implementation. Repository scripts generated the measurements; the author reviewed the method, results, and manuscript and accepts responsibility for the content.

References

[1] S. Ettinger, S. Cheng, B. Caine, C. Liu, H. Zhao, S. Pradhan, Y. Chai, B. Sapp, C. R. Qi, Y. Zhou, Z. Yang, A. Chouard, P. Sun, J. Ngiam, V. Vasudevan, A. McCauley, J. Shlens, and D. Anguelov, "Large scale interactive motion forecasting for autonomous driving: The Waymo open motion dataset," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9710–9719, Oct. 2021.

[2] NVIDIA, Y. Cao, R. de Lutio, S. Fidler, G. Garcia Cobo, Z. Gojcic, M. Igl, B. Ivanovic, P. Karkus, J. Martinez Esturo, M. Pavone, A. Smith, E. Tanimura, M. Tyszkiewicz, M. Watson, Q. Wu, and L. Zhang, "AlpaSim: A modular, lightweight, and data-driven research simulator for autonomous driving." Software, Oct. 2025.

[3] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, J. D. Co-Reyes, R. Agarwal, R. Roelofs, Y. Lu, N. Montali, P. Mougin, Z. Yang, B. White, A. Faust, R. McAllister, D. Anguelov, and B. Sapp, "Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. Datasets and Benchmarks Track.

[4] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, "NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. Datasets and Benchmarks Track.

[5] D. Dauner and Autonomous Vision Group, "NAVSIM baseline checkpoints." Hugging Face model repository, Sept. 2024. Commit 32d89c0; Apache-2.0.

[6] A. Sangiovanni-Vincentelli, W. Damm, and R. Passerone, "Taming Dr. Frankenstein: Contract-based design for cyber-physical systems," *European Journal of Control*, vol. 18, no. 3, pp. 217–238, 2012.

[7] L. de Alfaro and T. A. Henzinger, "Interface automata," in *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pp. 109–120, ACM, 2001.

[8] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, vol. 78 of *Proceedings of Machine Learning Research*, pp. 1–16, PMLR, 2017.

[9] N. Karnchanachari, D. Geromichalos, K. S. Tan, N. Li, C. Eriksen, S. Yaghoubi, N. Mehdipour, G. Bernasconi, W. K. Fong, Y. Guo, and H. Caesar, "Towards learning-based planning: The nuPlan benchmark for real-world autonomous driving," in *2024 IEEE International Conference on Robotics and Automation*, pp. 629–636, IEEE, May 2024.

[10] D. J. Fremont, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, and S. A. Seshia, "Scenic: A language for scenario specification and scene generation," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 63–78, ACM, 2019.

[11] T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, H. Ravanbakhsh, M. Vazquez-Chanlatte, and S. A. Seshia, "VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems," in *Computer Aided Verification*, vol. 11561 of *Lecture Notes in Computer Science*, pp. 432–442, Springer, 2019.

[12] S. Kim, M. Liu, J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, "DriveFuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1753–1767, ACM, Nov. 2022.

[13] Z. Wan, J. Shen, J. Chuang, X. Xia, J. Garcia, J. Ma, and Q. A. Chen, "Too afraid to drive: Systematic discovery of semantic DoS vulnerability in autonomous driving planning under physical-world attacks," in *Proceedings of the Network and Distributed System Security Symposium*, Internet Society, 2022.